φ-Calculus: Object-Oriented Formalism

Ver: 0.0.0

YEGOR BUGAYENKO, Huawei, Russia
MAXIM TRUNNIKOV, Huawei, Russia

Object-oriented programming (OOP) is one of the most popular paradigms used for building software systems[1]. However, despite its industrial and academic popularity, OOP is still missing a formal apparatus similar to λ-calculus, which functional programming is based on. A number of attempts were made to formalize OOP, but none of them managed to cover all the features available in modern OO programming languages, such as C++ or Java. We have made yet another attempt and created φ-calculus, intended as a minimal foundation on which such features may be built rather than a calculus that already covers them. This paper does not demonstrate the practical use or effect of φ-calculus but merely explains it.

Additional Key Words and Phrases: Object-Oriented Programming, Object Calculus

1 Introduction

Object-oriented programming (OOP) has become the dominant paradigm for software development, with languages such as Java, C++, Python, and C# being among the most widely used in industry [7, 29, 31, 32, 34, 41]. Despite this prevalence, the theoretical foundations of OOP remain less developed than those of functional programming [31, 37], which has long benefited from the λ-calculus [4, 9] as a formal basis. The absence of a comparable formalism for OOP hinders rigorous reasoning about object-oriented programs, complicates the development of verified compilers and static analyzers, and limits the potential for principled language design [20].

Several attempts have been made to formalize object-oriented (OO) concepts. Abadi and Cardelli [1] proposed an imperative object calculus, while Igarashi et al. [23] introduced Featherweight Java as a minimal core calculus. However, these formalisms either focus on specific language features or omit constructs commonly found in modern OO languages. As Nierstrasz [35] observed, the development of concurrent object-based programming languages has suffered from the lack of any generally accepted formal foundations.

This paper presents φ-calculus[2], a formalism designed to serve as a foundation for object-oriented programming. The calculus is built around a primitive—the object formation—which encapsulates formations, data, and functions. Objects interact through application, which attaches expressions to attributes, and dispatch, which retrieves attached expressions.

The contributions of this paper are as follows:

- A formal syntax for φ-calculus defined by a context-free grammar (Section 3).
- Rigorous definitions of the core constructs (Section 4).

[1] LaTeX sources of this paper are maintained in the REPOSITORY public GitHub repository, the rendered version is 0.0.0.

[2] Typeset with the `eolang` LaTeX package, version 0.26.4.

- A family of semantic operators (Section 5).
- A reference interpreter that runs φ-expressions[3].

The remainder of this paper is organized as follows. Section 2 provides an informal overview of the calculus. Section 3 presents the syntax as a context-free grammar. Section 4 defines the foundational concepts. Section 5 introduces the semantic operators. Section 6 discusses related work. Finally, Section 7 suggests directions of future studies.

2 Informal Overview

An object *formation* is a collection of *attributes*, which are uniquely named pairs, for example:

$$⟦\texttt{price} ↦ ∅, \texttt{color} ↦ ⟦Δ ↦ \texttt{FF-C0-CB}⟧⟧. \tag{1}$$

This formation has two attributes `price` and `color`. The formation is an *abstract formation* because the `price` attribute is *void*, i.e. there is nothing *attached* to it. The `color` attribute is attached to another formation with one Δ-*asset*, which is attached to *data* (three bytes).

A *full application* of a pair—an attribute and an *expression*—to an abstract formation *results* in a new *closed* formation, for example:

$$⟦\texttt{x} ↦ ∅⟧(\texttt{x} ↦ b) ⇝ ⟦\texttt{x} ↦ b⟧. \tag{2}$$

A *partial* application results in a new abstract formation, for example:

$$⟦\texttt{x} ↦ ∅, \texttt{y} ↦ ∅⟧(\texttt{x} ↦ b) ⇝ ⟦\texttt{x} ↦ b, \texttt{y} ↦ ∅⟧.$$

A formation may be *dispatched* from another formation with the help of *dot notation* where the right side *accesses* the left side, for example:

$$⟦\texttt{x} ↦ \overline{ξ.\texttt{p}(\texttt{t} ↦ b)}.\texttt{y}, \texttt{p} ↦ ⟦\texttt{t} ↦ ∅, \texttt{y} ↦ ξ.\texttt{t}⟧⟧.\texttt{x} ⇝ b. \tag{3}$$

The leftmost symbol ξ denotes the formation itself; the following `.p` part retrieves the formation attached to the attribute `p`; then, the application $(\texttt{t} ↦ b)$ makes a copy of the formation; finally, the `.y` part retrieves formation b.

Attributes are *immutable*, i.e. an application to a formation, where the attribute is already attached, results in a *terminator* denoted as ⊥ (error), for example:

$$⟦\texttt{x} ↦ b_1⟧(\texttt{x} ↦ b_2) ⇝ ⊥.$$

An expression may be textually reduced, for example:

$$\begin{aligned}⟦\texttt{x} ↦ ∅, \texttt{y} ↦ ∅⟧(\texttt{x} ↦ b_1)(\texttt{y} ↦ b_2) &⇝\\ ⇝ ⟦\texttt{x} ↦ b_1, \texttt{y} ↦ ∅⟧(\texttt{y} ↦ b_2) &⇝\\ ⇝ ⟦\texttt{x} ↦ b_1, \texttt{y} ↦ b_2⟧.\end{aligned}$$

The formation on the left is reduced to the formation on the right in two reduction steps. Some expressions may be in *normal form*, which means no further applicable reductions.

[3] https://github.com/objectionary/phino

$$
\begin{aligned}
\langle \text{Expression} \rangle &\to \langle \text{Formation} \rangle \mid \langle \text{Application} \rangle \mid \langle \text{Dispatch} \rangle \mid \langle \text{Locator} \rangle \mid \bot \\
\langle \text{Formation} \rangle &\to ⟦ \langle \text{Binding} \rangle ⟧ \\
\langle \text{Application} \rangle &\to \langle \text{Expression} \rangle \; (\; \langle \text{A-Pair} \rangle \;) \\
\langle \text{A-Pair} \rangle &\to \langle \tau\text{-Pair} \rangle \mid \langle \alpha\text{-Pair} \rangle \\
\langle \text{Dispatch} \rangle &\to \langle \text{Expression} \rangle \; . \; \langle \text{Attribute} \rangle \\
\langle \text{Locator} \rangle &\to \Phi \mid \xi \\
\langle \text{Binding} \rangle &\to \langle \text{Pair} \rangle \; \langle \text{Bindings} \rangle \mid \epsilon \\
\langle \text{Bindings} \rangle &\to , \; \langle \text{Pair} \rangle \; \langle \text{Bindings} \rangle \mid \epsilon \\
\langle \text{Pair} \rangle &\to \langle \varnothing\text{-Pair} \rangle \mid \langle \tau\text{-Pair} \rangle \mid \langle \Delta\text{-Pair} \rangle \mid \langle \lambda\text{-Pair} \rangle \\
\langle \varnothing\text{-Pair} \rangle &\to \langle \text{Attribute} \rangle \mapsto \varnothing \\
\langle \tau\text{-Pair} \rangle &\to \langle \text{Attribute} \rangle \mapsto \langle \text{Expression} \rangle \\
\langle \alpha\text{-Pair} \rangle &\to \langle \text{Alpha} \rangle \mapsto \langle \text{Expression} \rangle \\
\langle \Delta\text{-Pair} \rangle &\to \Delta \overset{\text{D}}{\mapsto} \langle \text{Data} \rangle \\
\langle \lambda\text{-Pair} \rangle &\to \lambda \overset{\text{D}}{\mapsto} \langle \text{Function} \rangle
\end{aligned}
$$

Fig. 1. Syntax as a context-free grammar, in BNF.

A formation is called a *decorator* if it has the φ-attribute with an expression attached to it, known as a *decoratee*. An attribute dispatched from a decorator reduces to the same attribute dispatched from the decoratee, if the attribute is not present in the decorator, for example:

$$⟦ \varphi \mapsto ⟦ \texttt{x} \mapsto b ⟧ ⟧.\texttt{x} \rightsquigarrow b.$$

A formation may have data attached only to its Δ-asset, for example:

$$⟦ \texttt{x} \mapsto ⟦ \Delta \overset{\text{D}}{\mapsto} \texttt{00-2A} ⟧ ⟧.$$

A formation may have a *function* attached to its λ-asset. Such a formation is referred to as an *atom*. An atom may be *morphed* to another expression by evaluating its function, for example:

$$\langle ⟦ \lambda \overset{\text{D}}{\mapsto} \texttt{Sqrt}, \texttt{x} \mapsto 256 ⟧, e, s \rangle \Rrightarrow \langle ⟦ \Delta \overset{\text{D}}{\mapsto} \texttt{10-} ⟧, s \rangle. \tag{4}$$

An expression may be *dataized* by morphing its atoms first and then extracting the data attached to the Δ-asset of the formation, for example:

$$\langle ⟦ \lambda \overset{\text{D}}{\mapsto} \texttt{Sqrt}, \texttt{x} \mapsto 256 ⟧, e, s \rangle \Downarrow \langle \texttt{16}, s \rangle.$$

3 Syntax

The syntax is defined by BNF in Fig. 1, where the starting symbol is ⟨Expression⟩.

Besides the literals mentioned in the grammar in orange, the alphabet includes three non-terminals that rewrite to terminals as follows:

- ⟨Attribute⟩: either 1) Greek letter φ, 2) Greek letter ρ, or 3) a string of lowercase English letters possibly with dashes inside, e.g. "`price`" or "`a-car`";
- ⟨Data⟩: a sequence of bytes in hexadecimal format, e.g. "`EF-41-5C`" is a sequence of three bytes, "`42-`" is a one-byte sequence (with a trailing dash to

Table 1. Syntax sugar.

Syntax sugar	Its more verbose canonical equivalent
$e(\tau_1 \mapsto e_1, \tau_2 \mapsto e_2, \ldots)$	$e(\tau_1 \mapsto e_1)(\tau_2 \mapsto e_2)\ldots$
$e(e_0, e_1, \ldots)$	$e(\alpha_0 \mapsto e_0, \alpha_1 \mapsto e_1, \ldots)$
$\tau_1(\tau_2, \tau_3, \ldots) \mapsto [\![B]\!]$	$\tau_1 \mapsto [\![\tau_2 \mapsto \varnothing, \tau_3 \mapsto \varnothing, \ldots, B]\!]$
$\tau_1 \mapsto \tau_2$	$\tau_1 \mapsto \xi.\tau_2$
$e{:}\tau$	$[\![\tau \mapsto e]\!]$
`"你好"`	Φ.`string`(Φ.`bytes`($[\![\Delta \mapsto$ `E4-BD-A0-E5-A5-BD`$]\!]$)) (UTF-8 string)
`42`	Φ.`number`(Φ.`bytes`($[\![\Delta \mapsto$ `40-45-00-00-00-00-00-00`$]\!]$)) (eight bytes per integer, assuming it has a floating point)
`3.14`	Φ.`number`(Φ.`bytes`($[\![\Delta \mapsto$ `40-09-1E-B8-51-EB-85-1F`$]\!]$)) (eight bytes per number with a floating point)
Φ.`nan`	Φ.`number`(Φ.`bytes`($[\![\Delta \mapsto$ `7F-F8-00-00-00-00-00-00`$]\!]$))
Φ.`pinf`	Φ.`number`(Φ.`bytes`($[\![\Delta \mapsto$ `7F-F0-00-00-00-00-00-00`$]\!]$)) (positive infinity)
Φ.`ninf`	Φ.`number`(Φ.`bytes`($[\![\Delta \mapsto$ `FF-F0-00-00-00-00-00-00`$]\!]$)) (negative infinity)

avoid confusion with integers), and "`--`" (double dash) is an empty sequence of bytes;

- ⟨Function⟩: a string of English letters and numbers where the first symbol is an uppercase letter, e.g. "`Sqrt`" or "`F1`";
- ⟨Alpha⟩: a Greek letter α with a non-negative whole-number index, e.g. α_2.

Table 1 shows all possible syntax sugar.

4 Foundations

In this section, we introduce the core concepts of φ-calculus. The purpose of this section is to define the entities that constitute the calculus—such as expressions, formations, and attributes—together with their structural relationships. The definitions given here are descriptive rather than operational. They explain what expressions are, not how they are evaluated or transformed.

Definition 4.1 (Expression). An **expression**, ranged over $\mathcal{E}$ by e_i, is a grammatical construct obeying the syntax of Fig. 1.

Definition 4.2 (Attribute). An **attribute**, ranged over $\mathcal{T}$ by τ_i, is an identifier to which either $\varnothing$ or an expression may be attached.

Definition 4.3 (Alpha). An **alpha** is a positional identifier.

Definition 4.4 (Void vs. Attached). An attribute is **void** if $\varnothing$ is attached to it, otherwise it is an **attached** attribute.

Definition 4.5 (Datum). A **datum**, ranged over $\mathcal{D}$ by δ_i, is a possibly empty sequence of 8-bit bytes.

Definition 4.6 (State). A **state** of evaluation s_i ranges over $\mathcal{S}$ and represents the external world—such as the store and input/output—that a function reads and shifts from one state to the next when applied, its internal structure left unspecified.

Definition 4.7 (Universe). A **universe** is an expression that serves as the global context of another expression, in which the Φ locator refers to the universe.

Definition 4.8 (Function). A **function** is a total mapping $\mathcal{B} \times \mathcal{E} \times \mathcal{S} \to \mathcal{E} \times \mathcal{S}$ that deterministically maps formation, universe, and state to an expression and a new state, ranged over $\mathcal{F}$ by f_i.

The implementation of functions is outside the scope of φ-calculus.

Definition 4.9 (Asset). An **asset** is an identifier to which either data (denoted as Δ-asset) or function (denoted as λ-asset) is attached.

Definition 4.10 (Binding). A **binding**, ranged over $\mathcal{G}$ by B_i, is a possibly empty sequence of key-value pairs, where all keys are unique.

Definition 4.11 (Formation). An object **formation**, denoted as $[\![B]\!]$, ranged over $\mathcal{B}$ by b_i, is a binding.

If $b = [\![B]\!]$, then the predicate $k \in b$ holds if the key k is present in B.

We introduce the term "formation" rather than using the more traditional term "construction" because the latter generally implies the presence of a class from which an object is being constructed or instantiated. Instead, formation is closer to the creation of a prototype, which may either be used "as is" or copied.

The following is an example of a formation with four pairs, where the first one is an asset attached to data, while the other three are attributes attached to expressions:

$$[\![\Delta \mapsto \texttt{00-2A}, \texttt{b} \mapsto b_2(\alpha_0 \mapsto b_3).\texttt{bar}, \texttt{a} \mapsto [\![\lambda \mapsto \texttt{Sqrt}]\!], \texttt{foo} \mapsto \bot]\!]. \tag{5}$$

The arrow $\mapsto$ denotes an attachment of an expression (right-hand side) to an attribute (left-hand side). The arrow $\mapsto$ denotes an attachment of data or function to an asset.

Definition 4.12 (Domain). A **domain** of binding B, denoted as $\bar{B}$, is a sequence of all attributes of B, excluding assets and ρ.

The domain of the binding in the formation of Eq. (5) is $\langle \texttt{b}, \texttt{a}, \texttt{foo}, \rangle$. Assets Δ and λ do not belong to the domain.

Definition 4.13 (Terminator). The **terminator**, denoted as $\bot$, is an expression that signals an error.

Definition 4.14 (Abstract Formation). A formation with at least one void attribute is an **abstract formation**.

Equation (1) is an example of an abstract formation, while the formation attached to its `color` attribute is closed.

A formation that is not an abstract formation may be called a *closed* formation.

Definition 4.15 (Application). An **application**, denoted as $e_1(\tau \mapsto e_2)$ or $e_1(\alpha_i \mapsto e_2)$, means an attempt to attach e_2 (the "argument") to either the τ attribute of e_1 (the "subject") or the i-th attribute of e_1.

Equation (2) demonstrates application, where $\texttt{x} \mapsto b$ is applied to an abstract formation $[\![\texttt{x} \mapsto \varnothing]\!]$. The application creates a new formation $[\![\texttt{x} \mapsto b]\!]$, while the existing formation remains intact.

The formation in Eq. (1) is an abstract formation, because its attribute `price` is void. The formation in Eq. (2) was an abstract formation before the application, but the formation created by the application is closed since its attribute `x` is not void (attached to b).

Even though $\varnothing$ being attached to an attribute of a formation resembles NULL references, there is a significant difference: in φ-calculus, void attributes may be attached to expressions only once, while any further reattachments are prohibited.

In $b(\alpha_i \mapsto e)$ application, e must be attached to the i-th attribute of b.

Definition 4.16 (Dispatch). A **dispatch**, denoted as $e.\tau$, where e is the "subject," means an attempt to retrieve what is attached to τ in the formation to which e may be transformed.

Definition 4.17 (Atom). An **atom** is a formation with a function attached to its λ-asset.

Equation (4) demonstrates an atom with a function that calculates the square root of a number, which it retrieves from the Δ-asset of $b.\alpha_0$ with the help of the morphing function (Section 5.5).

Definition 4.18 (Decorator). A formation is a **decorator** if it has an φ-attribute with an expression attached to it; the expression attached to the φ-attribute is its **decoratee**.

Definition 4.19 (Parent). Attaching expression e to the ρ attribute of formation b means setting the **parent** of b to e.

The presence of "parent" in each formation is essential for the coordination of inner formations after dispatch. Consider the following abstract formation with an inner formation:

$$[\![\texttt{x} \mapsto [\![\texttt{a} \mapsto \varnothing, \texttt{next} \mapsto [\![\varphi \mapsto \xi.\rho.\texttt{a.plus}(\texttt{1})]\!]]\!], \texttt{k} \mapsto \xi.\texttt{x}(\texttt{42})]\!].$$

Here, if the parent of $\Phi.\texttt{x.next}$ were attached in the formation, the result of dataization of $\Phi.\texttt{k.next}$ would not be equal to `43`. Instead, it would be equal to $\bot$, because $\Phi.\texttt{k.next}.\rho$ would still be attached to $\Phi.\texttt{x}$ after the dispatch of $\Phi.\texttt{k.next}$. The parent attribute may be compared with the `this` pointer in Java or C++, which does not point anywhere until a method of a class is called. Then, when the method is called, the `this` pointer refers to the formation that owns the method.

$$\mathsf{R}_{\text{CA}}\ \frac{\langle n_1, k\rangle \Rightarrow n_2 \qquad \langle e, k\rangle \Rightarrow n_3}{\langle n_1(\tau \mapsto e), k\rangle \Rightarrow n_2(\tau \mapsto n_3)} \qquad \mathsf{R}_{\text{CAA}}\ \frac{\langle n_1, k\rangle \Rightarrow n_2 \qquad \langle e, k\rangle \Rightarrow n_3}{\langle n_1(\alpha_i \mapsto e), k\rangle \Rightarrow n_2(\alpha_i \mapsto n_3)}$$

$$\mathsf{R}_{\text{CD}}\ \frac{\langle n_1, k\rangle \Rightarrow n_2}{\langle n_1.\tau, k\rangle \Rightarrow n_2.\tau} \qquad \mathsf{R}_{\text{CF}}\ \frac{}{\langle [\![B]\!], k\rangle \Rightarrow [\![B]\!]} \qquad \mathsf{R}_{\text{CG}}\ \frac{}{\langle \Phi, k\rangle \Rightarrow \Phi}$$

$$\mathsf{R}_{\text{CT}}\ \frac{}{\langle \bot, k\rangle \Rightarrow \bot} \qquad \mathsf{R}_{\text{CXI}}\ \frac{}{\langle \xi, k\rangle \Rightarrow k}$$

Fig. 2. Contextualization by induction, generated by phino 0.0.161.

5 Operators

In this section we introduce a family of semantic operators defined on φ-expressions. These operators do not extend the syntax of the calculus. Instead, they describe systematic transformations of expressions that reveal, modify, or project their semantic content. Each operator is defined in terms of the reduction semantics introduced earlier and preserves the core meaning of expressions while possibly changing their form or role.

5.1 Contextualization

Definition 5.1 (Absolute). An expression is **absolute**, denoted by k and ranging over $\mathcal{K} \subseteq \mathcal{E}$, if it is either a) Φ, b) formation, c) dispatch with an absolute subject, or d) application with absolute subject and argument.

Definition 5.2 (Contextualization). The *contextualization* function $\mathcal{E} \times \mathcal{K} \to \mathcal{E}$, denoted as $\langle e_1, k\rangle \Rightarrow e_2$ and specified by the rules of Fig. 2, turns an expression e_1 into e_2 by replacing with the absolute k every ξ that denotes the context of e_1 rather than that of a nested formation.

Theorem 5.3. *The contextualization function is total, meaning that every expression can be contextualized and its derivation terminates.*

5.2 Normalization

An expression that may be rewritten by the *rules* (or *reductions*) listed in Fig. 3 is a *reducible* expression. The notation $e_1 \rightsquigarrow e_2$, optionally followed by a condition, denotes a reduction of e_1 to e_2, if the condition holds.

A specific reduction may be denoted, for example, as $\rightsquigarrow_{\mathsf{R}_{\text{DOT}}}$, or just $\rightsquigarrow$ when no specific reduction is meant. The notation $e_1 \overset{*}{\rightsquigarrow} e_2$ denotes a reflexive transitive closure of all reductions, so that there is a possibly empty finite sequence of reductions between e_1 and e_2.

Definition 5.4 (Normal Form). An expression that has no more possible applications of reductions is *irreducible* or a *normal form*, denoted as n_i ranging over $\mathcal{N} \subseteq \mathcal{E}$.

$$
\begin{aligned}
\mathsf{R}_{\text{ALPHA}}&: \llbracket B_1, \tau \mapsto \varnothing, B_2 \rrbracket(\alpha_i \mapsto e) \rightsquigarrow \llbracket B_1, \tau \mapsto \varnothing, B_2 \rrbracket(\tau \mapsto e) \quad \text{if } i = |\overline{B_1}| \text{ and } \tau \neq \rho \\
\mathsf{R}_{\text{AMISS}}&: \llbracket B \rrbracket(\alpha_i \mapsto e) \rightsquigarrow \bot \quad \text{if } |\overline{B}| \leq i \\
\mathsf{R}_{\text{COPY}}&: \llbracket B_1, \tau \mapsto \varnothing, B_2 \rrbracket(\tau \mapsto k) \rightsquigarrow \llbracket B_1, \tau \mapsto k, B_2 \rrbracket \\
\mathsf{R}_{\text{DC}}&: \bot(\tau \mapsto e) \rightsquigarrow \bot \\
\mathsf{R}_{\text{DCA}}&: \bot(\alpha_i \mapsto e) \rightsquigarrow \bot \\
\mathsf{R}_{\text{DD}}&: \bot.\tau \rightsquigarrow \bot \\
\mathsf{R}_{\text{DL}}&: \llbracket B_1, \lambda \mapsto f, B_2 \rrbracket \rightsquigarrow \bot \quad \text{if } \Delta \in (B_1 \cup B_2) \\
\mathsf{R}_{\text{DOT}}&: \llbracket B_1, \tau \mapsto n, B_2 \rrbracket.\tau \rightsquigarrow e_1(\rho \mapsto e_2) \quad \text{if } [\Delta, \lambda] \not\subseteq (B_1 \cup B_2) \\
&\quad \text{if } \langle n, \llbracket B_1, B_2 \rrbracket \rangle \Rightarrow e_1 \text{ and } e_2 = \llbracket B_1, \tau \mapsto n, B_2 \rrbracket \\
\mathsf{R}_{\text{MISS}}&: \llbracket B \rrbracket(\tau \mapsto e) \rightsquigarrow \bot \quad \text{if } \tau \notin B \text{ and } \tau \neq \rho \\
\mathsf{R}_{\text{NORHO}}&: \llbracket B \rrbracket.\rho \rightsquigarrow \bot \quad \text{if } \rho \notin B \\
\mathsf{R}_{\text{NULL}}&: \llbracket B_1, \tau \mapsto \varnothing, B_2 \rrbracket.\tau \rightsquigarrow \bot \\
\mathsf{R}_{\text{OVER}}&: \llbracket B_1, \tau \mapsto e_1, B_2 \rrbracket(\tau \mapsto e_2) \rightsquigarrow \bot \quad \text{if } \tau \neq \rho \\
\mathsf{R}_{\text{OVERA}}&: \llbracket B_1, \tau \mapsto e_1, B_2 \rrbracket(\alpha_i \mapsto e_2) \rightsquigarrow \bot \quad \text{if } i = |\overline{B_1}| \text{ and } \tau \neq \rho \\
\mathsf{R}_{\text{SKIP}}&: \llbracket B \rrbracket(\rho \mapsto e) \rightsquigarrow \llbracket B \rrbracket \quad \text{if } \rho \notin B \\
\mathsf{R}_{\text{STAY}}&: \llbracket B_1, \rho \mapsto e_1, B_2 \rrbracket(\rho \mapsto e_2) \rightsquigarrow \llbracket B_1, \rho \mapsto e_1, B_2 \rrbracket \\
\mathsf{R}_{\text{STOP}}&: \llbracket B \rrbracket.\tau \rightsquigarrow \bot \quad \text{if } [\tau, \varphi, \lambda] \cap B = \emptyset
\end{aligned}
$$

Fig. 3. Reduction rules, generated by phino 0.0.161.

Thus, n is a normal form of e if $e \overset{*}{\rightsquigarrow} n$ and there is no expression e_1 such that $n \rightsquigarrow e_1$.

THEOREM 5.5. *Normalization is total: every expression has a normal form and its reduction terminates.*

Section A demonstrates how normalization works through examples.

THEOREM 5.6. *The reduction relation is confluent: if $e \overset{*}{\rightsquigarrow} e_1$ and $e \overset{*}{\rightsquigarrow} e_2$, then there exists an expression e_3 such that $e_1 \overset{*}{\rightsquigarrow} e_3$ and $e_2 \overset{*}{\rightsquigarrow} e_3$.*

Confluence guarantees that the normal form of an expression is unique, irrespective of the order in which the rules are applied.

We use Lean4 to automate the proof of Theorem 5.6, in the objectionary/proof GitHub repository.

5.3 Equivalence

Definition 5.7 (Equivalence). Two expressions are said to be syntactically equivalent or just **equivalent** (denoted by $\equiv$) if their normal forms are syntactically identical.

5.4 Evaluation

Definition 5.8 (Evaluation). The *evaluation* function $\mathcal{B} \times \mathcal{E} \times \mathcal{S} \rightharpoonup \mathcal{N} \times \mathcal{S}$, denoted as $\langle b, e, s_1 \rangle \longrightarrow \langle n, s_2 \rangle$ and specified by the rules of Fig. 4, maps an atom b to n,

$$\frac{e_1 \rightsquigarrow n}{\langle [\![B_1, \lambda \mapsto f, B_2]\!], e, s_1\rangle \longrightarrow \langle n, s_2\rangle} \text{ if } \langle e_1, s_2\rangle = f([\![B_1, \lambda \mapsto f, B_2]\!], e, s_1)$$

$$\frac{}{\langle [\![B]\!], e, s\rangle \longrightarrow \langle [\![B]\!], s\rangle} \text{ if } \lambda \notin B$$

Fig. 4. Evaluation rules.

calling by value the function attached to its λ-asset in the universe e, possibly shifting the state s_1 to s_2,

Because the function attached to an atom is arbitrary—its implementation lies outside the calculus (Section 4)—firing it may run any computation, and the calculus cannot guarantee that a datum ever comes back. Evaluation is therefore partial on two counts: the call may diverge, as when an atom dispatches back into its own object and recurs without bound, or it may halt without a value, as when the function aborts on an undefined operation such as division by zero. Morphing and dataization reach atoms and inherit that partiality, whereas contextualization and normalization fire no atom and are total (Theorems 5.3 and 5.5).

5.5 Morphing

Definition 5.9 (Morphing). The *morphing* function $\mathcal{N} \times \mathcal{E} \times \mathcal{S} \rightharpoonup (\mathcal{B} \cup \{\bot\}) \times \mathcal{S}$, denoted as $\langle n_1, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle$ and specified by the rules of Fig. 5, reduces a normal form n_1 to the formation it denotes, or to $\bot$ if it denotes none, by evaluating the atoms it reaches and resolving the Φ locator against the universe e, possibly shifting the state s_1 to s_2.

Morphing is a partial function, since the atoms it reaches (Section 5.4) may fire without bound, and it may follow a cyclic decoration chain that never resolves.

Theorem 5.10. *The morphing function is confluent, meaning that every normal form can be morphed to at most one formation or* $\bot$.

5.6 Dataization

Definition 5.11 (Dataization). The *dataization* function $\mathcal{N} \times \mathcal{E} \times \mathcal{S} \rightharpoonup \mathcal{D} \times \mathcal{S}$, denoted as $\langle n, e, s_1\rangle \Downarrow \langle \delta, s_2\rangle$ and specified by the rules of Fig. 6, extracts the datum δ that a normal form n produces by morphing n to a formation in the universe e and reading the data attached to its Δ-asset, possibly shifting the state s_1 to s_2.

Theorem 5.12 (Progress). *At least one rule of Fig. 6 applies to every normal form except* $\bot$*, which matches none, so a dataization derivation halts only at a datum or at* $\bot$*, the error that* $\bot$ *signals.*

Theorem 5.13. *The dataization function is confluent, meaning that every normal form can be dataized to at most one datum.*

Section B demonstrates how the dataization function works through examples.

$$\mathsf{R}_{\text{DEAD}}\ \frac{}{\langle \bot, e, s\rangle \Rrightarrow \langle \bot, s\rangle}$$

$$\mathsf{R}_{\text{MA}}\ \frac{\langle n_1, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle \qquad n_2(\tau \mapsto k) \rightsquigarrow n_3 \qquad \langle n_3, e, s_2\rangle \Rrightarrow \langle n_4, s_3\rangle}{\langle n_1(\tau \mapsto k), e, s_1\rangle \Rrightarrow \langle n_4, s_3\rangle}$$

$$\mathsf{R}_{\text{MAA}}\ \frac{\langle n_1, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle \qquad n_2(\alpha_i \mapsto k) \rightsquigarrow n_3 \qquad \langle n_3, e, s_2\rangle \Rrightarrow \langle n_4, s_3\rangle}{\langle n_1(\alpha_i \mapsto k), e, s_1\rangle \Rrightarrow \langle n_4, s_3\rangle}$$

$$\mathsf{R}_{\text{MAAD}}\ \frac{\langle \bot, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle}{\langle n(\alpha_i \mapsto n_1), e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle}\ \text{if } n_1 \notin \mathcal{K}$$

$$\mathsf{R}_{\text{MAD}}\ \frac{\langle \bot, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle}{\langle n(\tau \mapsto n_1), e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle}\ \text{if } n_1 \notin \mathcal{K}$$

$$\mathsf{R}_{\text{MD}}\ \frac{\langle n_1, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle \qquad n_2.\tau \rightsquigarrow n_3 \qquad \langle n_3, e, s_2\rangle \Rrightarrow \langle n_4, s_3\rangle}{\langle n_1.\tau, e, s_1\rangle \Rrightarrow \langle n_4, s_3\rangle}\ \text{if } n_1 \notin \mathcal{B}$$

$$\mathsf{R}_{\text{MF}}\ \frac{}{\langle [\![B]\!], e, s\rangle \Rrightarrow \langle [\![B]\!], s\rangle} \qquad \mathsf{R}_{\text{MG}}\ \frac{\langle \bot, \Phi, s_1\rangle \Rrightarrow \langle n, s_2\rangle}{\langle \Phi, \Phi, s_1\rangle \Rrightarrow \langle n, s_2\rangle}$$

$$\mathsf{R}_{\lambda}\ \frac{\langle [\![B]\!], e, s_1\rangle \longrightarrow \langle n_1, s_2\rangle \qquad n_1.\tau \rightsquigarrow n_2 \qquad \langle n_2, e, s_2\rangle \Rrightarrow \langle n_3, s_3\rangle}{\langle [\![B]\!].\tau, e, s_1\rangle \Rrightarrow \langle n_3, s_3\rangle}\ \text{if } \lambda \in B$$

$$\mathsf{R}_{\varphi}\ \frac{[\![B]\!].\varphi.\tau \rightsquigarrow n_1 \qquad \langle n_1, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle}{\langle [\![B]\!].\tau, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle}\ \text{if } \varphi \in B \text{ and } [\tau, \lambda] \cap B = \emptyset$$

$$\mathsf{R}_{\Phi}\ \frac{e \rightsquigarrow n_1 \qquad \langle n_1, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle}{\langle \Phi, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle}\ \text{if } e \neq \Phi \qquad \mathsf{R}_{\xi}\ \frac{\langle \bot, e, s_1\rangle \Rrightarrow \langle n, s_2\rangle}{\langle \xi, e, s_1\rangle \Rrightarrow \langle n, s_2\rangle}$$

Fig. 5. Morphing rules, generated by phino 0.0.161.

$$\mathsf{R}_{\text{BOX}}\ \frac{[\![B]\!].\varphi \rightsquigarrow n \qquad \langle n, e, s_1\rangle \Downarrow \langle \delta, s_2\rangle}{\langle [\![B]\!], e, s_1\rangle \Downarrow \langle \delta, s_2\rangle}\ \text{if } \varphi \in B \text{ and } [\Delta, \lambda] \cap B = \emptyset$$

$$\mathsf{R}_{\Delta}\ \frac{}{\langle [\![B_1, \Delta \mapsto \delta, B_2]\!], e, s\rangle \Downarrow \langle \delta, s\rangle}\ \text{if } [\lambda] \cap (B_1 \cup B_2) = \emptyset$$

$$\mathsf{R}_{\text{FIRE}}\ \frac{\langle [\![B]\!], e, s_1\rangle \longrightarrow \langle n, s_2\rangle \qquad \langle n, e, s_2\rangle \Downarrow \langle \delta, s_3\rangle}{\langle [\![B]\!], e, s_1\rangle \Downarrow \langle \delta, s_3\rangle}\ \text{if } \lambda \in B \text{ and } [\Delta] \cap B = \emptyset$$

$$\mathsf{R}_{\text{NONE}}\ \frac{\langle \bot, e, s_1\rangle \Downarrow \langle \delta, s_2\rangle}{\langle [\![B]\!], e, s_1\rangle \Downarrow \langle \delta, s_2\rangle}\ \text{if } [\Delta, \lambda, \varphi] \cap B = \emptyset$$

$$\mathsf{R}_{\text{NORM}}\ \frac{\langle n_1, e, s_1\rangle \Rrightarrow \langle n_2, s_2\rangle \qquad \langle n_2, e, s_2\rangle \Downarrow \langle \delta, s_3\rangle}{\langle n_1, e, s_1\rangle \Downarrow \langle \delta, s_3\rangle}\ \text{if } n_1 \notin \mathcal{B} \text{ and } n_1 \neq \bot$$

Fig. 6. Dataization rules, generated by phino 0.0.161.

5.7 Congruence

Definition 5.14 (Congruence). Two expressions n_1 and n_2 are said to be behaviorally equivalent or **congruent** (denoted by $\cong$) if for any state s_1 and any e their dataizations are either both undefined, or both defined with the same datum and state, i.e. there exist δ and s_2 such that:

$$\langle n_1, e, s_1 \rangle \Downarrow \langle \delta, s_2 \rangle \quad \text{and} \quad \langle n_2, e, s_1 \rangle \Downarrow \langle \delta, s_2 \rangle.$$

Dataization gives the calculus its intended, observational semantics: the meaning of an expression is the datum it computes under a given universe and state, and two objects denote the same behavior exactly when they are congruent. The reduction rules are sound with respect to this meaning, since equivalent expressions are congruent ($\equiv$ implies $\cong$)—identical normal forms dataize identically—so normalizing an expression never changes what it computes.

Two congruent expressions may be non-equivalent. For example, $e_1 \cong e_2$ holds while $e_1 \equiv e_2$ doesn't hold for the following expressions:

$$e_1 = [\![\texttt{foo} \mapsto \varnothing, \Delta \mapsto \texttt{01-02}]\!] \quad e_2 = [\![\texttt{bar} \mapsto \varnothing, \Delta \mapsto \texttt{01-02}]\!].$$

6 Related Work

In this section we analyze and categorize prior art related to our work. Neither object-oriented formalism nor pure object-oriented languages are new research topics. However, we identified certain gaps in existing studies that make us believe that our work has novelty.

Attempts were made to formalize OOP and introduce an object calculus similar to the lambda calculus [4] used in functional programming. For example, Abadi and Cardelli [1] suggested an imperative calculus of objects, which was extended by Bono and Fisher [6] to support classes, by Gordon and Hankin [21] to support concurrency and synchronization, and by Jeffrey [26] to support distributed programming. These calculi, together with their functional counterpart, were consolidated into a comprehensive theory of objects [2].

A parallel line of work embeds objects directly into the λ-calculus rather than defining a standalone calculus. Mitchell et al. [33] introduced a lambda calculus of objects with method specialization, extended by Di Gianantonio et al. [15] with self-inflicted extension. These systems show that object features can be encoded on a functional core, which qualifies the recurring claim that OOP lacks a λ-calculus of its own; φ-calculus takes the opposite route, treating the object rather than the function as its single primitive, so that dispatch and decoration are native constructs rather than encodings.

Earlier, Honda and Tokoro [22] combined OOP and π-calculus in order to introduce object calculus for asynchronous communication, which was further referenced by Jones [27] in their work on object-based design notation.

A few attempts were made to reduce existing OOP languages and formalize what is left. Featherweight Java is the most notable example proposed by Igarashi et al. [23], which omits almost all features of the full language (including interfaces and

even assignment) to obtain a small calculus. Later it was extended by Jagannathan et al. [24] to support nested and multi-threaded transactions. Featherweight Java is used in formal languages such as Obsidian [13] and SJF [40].

The calculi above are class-based, whereas φ-calculus is prototype-based: objects arise by copying and decorating other objects rather than by instantiating classes. This tradition begins with Self [39], whose delegation mechanism Stein [38] showed to subsume inheritance, and continues in calculi of delegation by object composition [5]. The meaning of inheritance in such settings has been given denotationally by Cook and Palsberg [14] as a fixed point over records; φ-calculus instead renders it operationally, through the φ-decoration that its morphing operator resolves (Section 5.5).

Another example is Larch/C++ [8], which is a formal algebraic interface specification language tailored to C++. It allows interfaces of C++ classes and functions to be documented in a way that is unambiguous and concise.

Several attempts to formalize OOP were made by extensions of the most popular formal notations and methods, such as Object-Z [17] and VDM++ [18]. In Object-Z, state and operation schemes are encapsulated into classes. The formal model is based upon the idea of a class history [16]. However, all these OO extensions do not have comprehensive refinement rules that can be used to transform specifications into implemented code in an actual OO programming language, as was noted by Paige and Ostroff [36].

Bancilhon and Khoshafian [3] suggested an object calculus as an extension to relational calculus. Jankowska [25] further developed these ideas and related them to a Boolean algebra. Lee et al. [30] developed an algorithm to transform an object calculus into an object algebra.

However, all these theoretical attempts to formalize OO languages were unable to fully describe their features, as was noted by Nierstrasz [35]: "The development of concurrent object-based programming languages has suffered from the lack of any generally accepted formal foundations for defining their semantics." The visual and architectural notations often used to model object-oriented designs fare worse still: Eden [19], surveying them, observed that "Not one of the notations is defined formally, nor provided with denotational semantics, nor founded on axiomatic semantics." Moreover, Ciaffaglione et al. [10, 11, 12] noted in their series of works that, at the time, relatively little formal work had been carried out on object-based languages. Mechanized treatments have appeared since—among them a machine-checked model of a Java-like language, virtual machine, and compiler [28]—yet a compact object calculus whose own meta-theory is machine-checked remains uncommon.

What sets φ-calculus apart from the surveyed formalisms is therefore not broader feature coverage, which it deliberately forgoes (Section 7), but this uniformity together with a confluence result certified in Lean 4 (Theorem 5.6).

7 Future Studies

This paper intentionally focuses on defining φ-calculus (syntax, foundations, and semantic operators) and does not yet demonstrate practical use. At the same time, it motivates the calculus as a step toward more rigorous reasoning, verified compilers

and static analyzers, and better language design. Future work may build on φ-calculus by creating the following artifacts:

- A short list of core laws about reductions (what always works, what may fail).
- A clear way to prove that two expressions behave the same in any context.
- A type system that catches mistakes early.
- A way to mark which expressions are pure and which may have side effects.
- A catalog of safe rewrites (how to change code without changing meaning).
- A translation from Java or C++ into φ-expressions.
- A standard library of built-in atoms with precise rules.
- Larger language features built as add-ons (overriding, interfaces, constructors).
- Add-ons for mutation, concurrency, and distribution, with simple safety guarantees.

We invite readers to extend, formalize, and apply φ-calculus to real object-oriented patterns and languages, and to help turn these foundations into a shared body of results and artifacts.

8 Acknowledgments

Many thanks to (in alphabetical order of last names) Aliaksei Bialíauski, Fabricio Cabral, Kirill Chernyavskiy, Piotr Chmielowski, Danilo Danko, Konstantin Gukov, Andrew Graur, Ali-Sultan Kigizbaev, Nikolai Kudasov, Alexander Legalov, Mikhail Lipanin, Tymur λysenko, Alexandr Naumchev, Alonso A. Ortega, John Page, Alex Panov, Maxim Petrov, Alexander Pushkarev, Marcos Douglas B. Santos, Alex Semenyuk, Violetta Sim, Sergei Skliar, Stian Soiland-Reyes, Viacheslav Tradunskyi, Ilya Trub, César Soto Valero, Alena Vasileva, David West, Vladimir Zakharov, and Evgeny Zouev for their contribution to the development of φ-calculus.

References

[1] Martín Abadi and Luca Cardelli. 1995. An Imperative Object Calculus. *Theory and Practice of Object Systems* 1, 3 (1995). doi:10.1002/tapo.4410010304

[2] Martín Abadi and Luca Cardelli. 1996. *A Theory of Objects*. Springer.

[3] Francois Bancilhon and Setrag Khoshafian. 1985. A Calculus for Complex Objects. In *Proceedings of the Symposium on Principles of Database Systems*. doi:10.1145/325405.325451

[4] Hendrik P. Barendregt. 2012. *The Lambda Calculus: Its Syntax and Semantics*. College Publications.

[5] Lorenzo Bettini, Viviana Bono, and Betti Venneri. 2011. Delegation by Object Composition. *Science of Computer Programming* 76, 1 (2011). doi:10.1016/j.scico.2010.04.006

[6] Viviana Bono and Kathleen Fisher. 1998. An Imperative, First-Order Calculus With Object Extension. In *Proceedings of the European Conference on Object-Oriented Programming*. doi:10.1007/bfb0054104

[7] Grady Booch, Robert A. Maksimchuk, Michael W. Engle, Bobbi J. Young, Jim Connallen, and Kelli A. Houston. 1994. *Object-Oriented Analysis and Design With Applications*. Addison-Wesley. doi:10.5555/1407387

[8] Yoonsik Cheon and Gary T. Leavens. 1994. A Quick Overview of Larch/C++.

[9] Alonzo Church and J. Barkley Rosser. 1936. Some Properties of Conversion. *Transactions of the American Mathematical Society* 39, 3 (1936), 472–482. doi:10.2307/2268572

[10] Alberto Ciaffaglione, Luigi Liquori, and Marino Miculan. 2003. Imperative

Object-Based Calculi in Co-Inductive Type Theories. In *Proceedings of the International Conference on Logic for Programming Artificial Intelligence and Reasoning*. doi:10.1007/978-3-540-39813-4_18

[11] Alberto Ciaffaglione, Luigi Liquori, and Marino Miculan. 2003. Reasoning on an Imperative Object-Based Calculus in Higher Order Abstract Syntax. In *Proceedings of the MERLIN'03: Proceedings of the Workshop on Mechanized Reasoning About Languages With Variable Binding*. doi:10.1145/976571.976574

[12] Alberto Ciaffaglione, Luigi Liquori, and Marino Miculan. 2007. Reasoning About Object-Based Calculi in (Co) Inductive Type Theory and the Theory of Contexts. *Journal of Automated Reasoning* 39, 1 (2007). doi:10.1007/s10817-007-9077-x

[13] Michael J. Coblenz, Reed Oei, Tyler Etzel, Paulette Koronkevich, Miles Baker, Yannick Bloem, Brad A. Myers, Joshua Sunshine, and Jonathan Aldrich. 2020. Obsidian: Typestate and Assets for Safer Blockchain Programming. *ACM Transactions on Programming Languages and Systems* 42, 3 (2020). doi:10.1145/3417516

[14] William R. Cook and Jens Palsberg. 1989. A Denotational Semantics of Inheritance and Its Correctness. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications*. 433–443. doi:10.1145/74877.74922

[15] Pietro Di Gianantonio, Furio Honsell, and Luigi Liquori. 1998. A Lambda Calculus of Objects With Self-Inflicted Extension. In *Proceedings of the Conference on Object-Oriented Programming, Systems, Languages, and Applications*. doi:10.1145/286936.286947

[16] David Duke and Roger Duke. 1990. Towards a Semantics for Object-Z. In *Proceedings of the International Symposium of VDM Europe*. doi:10.1007/3-540-53692-X_56

[17] Roger Duke, Paul King, Gordon Rose, and Graeme Smith. 2000. *The Object-Z Specification Language*. Citeseer.

[18] Eugene Durr and Jan Van Katwijk. 1992. VDM++, a Formal Specification Language for Object-Oriented Designs. In *Proceedings of the Computer Systems and Software Engineering*. doi:10.1109/CSSE.1992.209289

[19] Amnon Eden. 2002. A Visual Formalism for Object-Oriented Architecture.

[20] Amnon H. Eden and Yoram Hirshfeld. 2001. Principles in Formal Specification of Object-Oriented Architectures. In *Proceedings of the CASCON'01: Proceedings of the Conference of the Centre for Advanced Studies on Collaborative Research*. doi:10.5555/782096.782099

[21] Andy Gordon and Paul D. Hankin. 1998. A Concurrent Object Calculus: Reduction and Typing.

[22] Kohei Honda and Mario Tokoro. 1991. An Object Calculus for Asynchronous Communication. In *Proceedings of the European Conference on Object-Oriented Programming*. doi:10.1007/BFb0057019

[23] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. 2001. Featherweight Java: A Minimal Core Calculus for Java and GJ. *ACM Transactions on Programming Languages and Systems* 23, 3 (2001). doi:10.1145/503502.503505

[24] Suresh Jagannathan, Jan Vitek, Adam Welc, and Antony Hosking. 2005. A Transactional Object Calculus. *Science of Computer Programming* 57, 2 (2005). doi:10.1016/j.scico.2005.02.008

[25] Beata Jankowska. 2003. Yet Another Object-Oriented Data Model and Its Application. *Control and Cybernetics* 32, 2 (2003). doi:10.3166/ejc.32.123-142

[26] Alan Jeffrey. 1999. A Distributed Object Calculus. In *Proceedings of the International Workshop on Foundations of Object-Oriented Languages*. doi:10.1145/319195.319216

[27] Cliff B. Jones. 1993. A Pi-Calculus Semantics for an Object-Based Design Notation. In *Proceedings of the International Conference on Concurrency Theory*. doi:10.1007/3-540-57208-2_6

[28] Gerwin Klein and Tobias Nipkow. 2006. A Machine-Checked Model for a Java-Like Language, Virtual Machine, and Compiler.

ACM Transactions on Programming Languages and Systems 28, 4 (2006), 619–695. doi:10.1145/1146809.1146811

[29] Tim Korson and John D. McGregor. 1990. Understanding Object-Oriented: A Unifying Paradigm. *Communications of the ACM* 33, 9 (1990), 40–60. doi:10.1145/83880.84459

[30] Kwak Lee, Hong-Ro Ryu, and Keun-Ho Hoon-Sung. 1996. A Translation of an Object Calculus into an Object Algebra. *The Transactions of the Korea Information Processing Society* 3, 4 (1996). doi:10.3745/KIPST.1996.3.4.123

[31] Ole Lehrmann Madsen and Birger Møller-Pedersen. 1988. What Object-Oriented Programming May Be — And What It Does Not Have to Be. In *Proceedings of the European Conference on Object-Oriented Programming*. Springer, 1–20. doi:10.1007/3-540-45910-3_1

[32] Bertran Meyer. 1997. *Object-Oriented Software Construction* (2 ed.). Prentice Hall. doi:10.5555/261119

[33] John C. Mitchell, Furio Honsell, and Kathleen Fisher. 1993. A Lambda Calculus of Objects and Method Specialization. In *Proceedings of the Symposium on Logic in Computer Science*. IEEE. doi:10.1109/LICS.1993.287566

[34] Oscar Nierstrasz. 1989. A Survey of Object-Oriented Concepts.

[35] Oscar Nierstrasz. 1991. Towards an Object Calculus. In *Proceedings of the European Conference on Object-Oriented Programming*. doi:10.1007/BFb0057013

[36] Richard F. Paige and Jonathan S. Ostroff. 1999. An Object-Oriented Refinement Calculus.

[37] Tim Rentsch. 1982. Object Oriented Programming. *ACM SIGPLAN Notices* 17, 9 (1982), 51–57. doi:10.1145/947955.947961

[38] Lynn Andrea Stein. 1987. Delegation Is Inheritance. *ACM SIGPLAN Notices* 22, 12 (1987), 138–146. doi:10.1145/38807.38820

[39] David Ungar and Randall B. Smith. 1987. Self: The Power of Simplicity. In *Proceedings of the SIGPLAN Notices*, Vol. 22. 227–242. doi:10.1145/38807.38828

[40] Artem Usov and Prnela Dardha. 2020. SJF: an Implementation of Semantic Featherweight Java. In *Proceedings of the Coordination Models and Languages (COORDINATION)*, Vol. 12134. doi:10.1007/978-3-030-50029-0_10

[41] David West. 2004. *Object Thinking*. Pearson Education. doi:10.5555/984130

A Examples of Normalization

The following examples demonstrate how the reduction rules of Fig. 3 may normalize some expressions, involving the contextualization function (Section 5.1):

$$\overbrace{\llbracket \mathtt{x} \mapsto \mathtt{t}, \mathtt{t} \mapsto \varnothing \rrbracket}^{\Pi_1}.\mathtt{x} \rightsquigarrow_{\mathrm{R_{DOT}}} \rightsquigarrow \varnothing{:}\mathtt{t}.\mathtt{t}(\rho \mapsto \Pi_1) \rightsquigarrow_{\mathrm{R_{NULL}}} \rightsquigarrow \bot(\rho \mapsto \Pi_1) \rightsquigarrow_{\mathrm{R_{DC}}} \rightsquigarrow \bot.$$

$$\overbrace{42.0{:}\mathtt{t}}^{\Pi_2}.\mathtt{t}{:}\mathtt{x}.\mathtt{x} \rightsquigarrow_{\mathrm{R_{DOT}}} \rightsquigarrow 42.0{:}\mathtt{x}.\mathtt{x} \rightsquigarrow_{\mathrm{R_{DOT}}} \rightsquigarrow 42.0.$$

$$\varnothing{:}\mathtt{x}(\alpha_1 \mapsto 42.0).\mathtt{x} \rightsquigarrow_{\mathrm{R_{AMISS}}} \rightsquigarrow \bot.\mathtt{x} \rightsquigarrow_{\mathrm{R_{DD}}} \rightsquigarrow \bot.$$

$$\varnothing{:}\mathtt{x}(42.0).\mathtt{x} \rightsquigarrow_{\mathrm{R_{ALPHA}}} \rightsquigarrow \varnothing{:}\mathtt{x}(\mathtt{x} \mapsto 42.0).\mathtt{x} \rightsquigarrow_{\mathrm{R_{COPY}}} \rightsquigarrow 42.0{:}\mathtt{x}.\mathtt{x} \rightsquigarrow_{\mathrm{R_{DOT}}} \rightsquigarrow 42.0.$$

$$\llbracket \mathtt{x} \mapsto \mathtt{Fn}{:}\lambda.\rho.\mathtt{k}, \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathrm{R_{NORHO}}} \rightsquigarrow \llbracket \mathtt{x} \mapsto \bot.\mathtt{k}, \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\mathrm{R_{DD}}} \rightsquigarrow \overbrace{\llbracket \mathtt{x} \mapsto \bot, \mathtt{k} \mapsto 42.0 \rrbracket}^{\Pi_3}.\mathtt{x} \rightsquigarrow_{\mathrm{R_{DOT}}} \rightsquigarrow \bot(\rho \mapsto \Pi_3) \rightsquigarrow_{\mathrm{R_{DC}}} \rightsquigarrow \bot.$$

$$\llbracket \mathtt{x} \mapsto \varnothing, \mathtt{y} \mapsto \mathtt{x} \rrbracket(\mathtt{x} \mapsto 42.0).\mathtt{y} \rightsquigarrow_{\mathrm{R_{COPY}}} \rightsquigarrow \overbrace{\llbracket \mathtt{x} \mapsto 42.0, \mathtt{y} \mapsto \mathtt{x} \rrbracket}^{\Pi_4}.\mathtt{y} \rightsquigarrow_{\mathrm{R_{DOT}}} \rightsquigarrow 42.0{:}\mathtt{x}.\mathtt{x}(\rho \mapsto \Pi_4) \rightsquigarrow_{\mathrm{R_{DOT}}} \rightsquigarrow 42.0.$$

$$\llbracket \mathtt{x} \mapsto \mathtt{t}, \mathtt{t} \mapsto \varnothing \rrbracket(\mathtt{t} \mapsto 42.0) \rightsquigarrow_{\mathrm{R_{COPY}}} \rightsquigarrow \llbracket \mathtt{x} \mapsto \mathtt{t}, \mathtt{t} \mapsto 42.0 \rrbracket.$$

$$\llbracket \mathtt{x} \mapsto \mathtt{t}, \varphi \mapsto \llbracket \rrbracket{:}\mathtt{t} \rrbracket.\mathtt{x} \rightsquigarrow_{\mathrm{R_{DOT}}} \rightsquigarrow \llbracket \rrbracket{:}\mathtt{t}{:}\varphi.\mathtt{t}(\rho \mapsto \llbracket \mathtt{x} \mapsto \mathtt{t}, \varphi \mapsto \llbracket \rrbracket{:}\mathtt{t} \rrbracket).$$

$$\overbrace{\rho.\rho.\mathtt{t}{:}\mathtt{x}}^{\Pi_5}.\mathtt{x} \rightsquigarrow_{\mathrm{R_{DOT}}} \rightsquigarrow \llbracket \rrbracket.\rho.\rho.\mathtt{t}(\rho \mapsto \Pi_5) \rightsquigarrow_{\mathrm{R_{NORHO}}} \rightsquigarrow \bot.\rho.\mathtt{t}(\rho \mapsto \Pi_5) \rightsquigarrow_{\mathrm{R_{DD}}} \rightsquigarrow \bot.\mathtt{t}(\rho \mapsto \Pi_5) \rightsquigarrow_{\mathrm{R_{DD}}} \rightsquigarrow \bot(\rho \mapsto \Pi_5) \rightsquigarrow_{\mathrm{R_{DC}}} \rightsquigarrow \bot.$$

$$
\begin{array}{l}
\llbracket \mathtt{x} \mapsto \llbracket \rho \mapsto \varnothing \rrbracket.\rho.\mathtt{k},\ \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{NULL}}} \\
\quad \rightsquigarrow \llbracket \mathtt{x} \mapsto \bot.\mathtt{k},\ \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DD}}} \\
\quad \rightsquigarrow \overbrace{\llbracket \mathtt{x} \mapsto \bot,\ \mathtt{k} \mapsto 42.0 \rrbracket}^{\Pi_6}.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
\quad \rightsquigarrow \bot(\rho \mapsto \Pi_6) \rightsquigarrow_{\text{R}_{\text{DC}}} \\
\quad \rightsquigarrow \bot.
\end{array}
$$

$$
\begin{array}{l}
\overbrace{\llbracket \mathtt{x} \mapsto \mathtt{t}(\mathtt{k} \mapsto \llbracket\rrbracket).\mathtt{k},\ \mathtt{t}(\mathtt{k}) \mapsto \llbracket\rrbracket \rrbracket}^{\Pi_7}.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
\quad \rightsquigarrow \llbracket \mathtt{t}(\mathtt{k}) \mapsto \llbracket\rrbracket \rrbracket.\mathtt{t}(\mathtt{k} \mapsto \llbracket\rrbracket).\mathtt{k}(\rho \mapsto \Pi_7) \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
\quad \rightsquigarrow \varnothing{:}\mathtt{k}(\rho \mapsto \llbracket \mathtt{t}(\mathtt{k}) \mapsto \llbracket\rrbracket \rrbracket,\ \mathtt{k} \mapsto \llbracket\rrbracket).\mathtt{k}(\rho \mapsto \Pi_7) \rightsquigarrow_{\text{R}_{\text{SKIP}}} \\
\quad \rightsquigarrow \varnothing{:}\mathtt{k}(\mathtt{k} \mapsto \llbracket\rrbracket).\mathtt{k}(\rho \mapsto \Pi_7) \rightsquigarrow_{\text{R}_{\text{COPY}}} \\
\quad \rightsquigarrow \llbracket\rrbracket{:}\mathtt{k}.\mathtt{k}(\rho \mapsto \Pi_7) \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
\quad \rightsquigarrow \llbracket\rrbracket(\rho \mapsto \llbracket\rrbracket{:}\mathtt{k},\ \rho \mapsto \Pi_7) \rightsquigarrow_{\text{R}_{\text{SKIP}}} \\
\quad \rightsquigarrow \llbracket\rrbracket(\rho \mapsto \Pi_7) \rightsquigarrow_{\text{R}_{\text{SKIP}}} \\
\quad \rightsquigarrow \llbracket\rrbracket.
\end{array}
$$

$$
\begin{array}{l}
\overbrace{\llbracket \mathtt{x} \mapsto \varnothing{:}\mathtt{t}(\mathtt{t} \mapsto \mathtt{k}),\ \mathtt{k} \mapsto \mathtt{Fn}{:}\lambda \rrbracket}^{\Pi_8}.\mathtt{x}.\mathtt{t}.\mathtt{p}{:}\mathtt{a} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
\quad \rightsquigarrow \varnothing{:}\mathtt{t}(\mathtt{t} \mapsto \mathtt{Fn}{:}\lambda{:}\mathtt{k}.\mathtt{k},\ \rho \mapsto \Pi_8).\mathtt{t}.\mathtt{p}{:}\mathtt{a} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
\quad \rightsquigarrow \varnothing{:}\mathtt{t}(\mathtt{t} \mapsto \mathtt{Fn}{:}\lambda(\rho \mapsto \mathtt{Fn}{:}\lambda{:}\mathtt{k}),\ \rho \mapsto \Pi_8).\mathtt{t}.\mathtt{p}{:}\mathtt{a} \rightsquigarrow_{\text{R}_{\text{SKIP}}} \\
\quad \rightsquigarrow \varnothing{:}\mathtt{t}(\mathtt{t} \mapsto \mathtt{Fn}{:}\lambda,\ \rho \mapsto \Pi_8).\mathtt{t}.\mathtt{p}{:}\mathtt{a} \rightsquigarrow_{\text{R}_{\text{COPY}}} \\
\quad \rightsquigarrow \overbrace{\mathtt{Fn}{:}\lambda{:}\mathtt{t}}^{\Pi_9}(\rho \mapsto \Pi_8).\mathtt{t}.\mathtt{p}{:}\mathtt{a} \rightsquigarrow_{\text{R}_{\text{SKIP}}} \\
\quad \rightsquigarrow \Pi_9.\mathtt{t}.\mathtt{p}{:}\mathtt{a} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
\quad \rightsquigarrow \mathtt{Fn}{:}\lambda(\rho \mapsto \Pi_9).\mathtt{p}{:}\mathtt{a} \rightsquigarrow_{\text{R}_{\text{SKIP}}} \\
\quad \rightsquigarrow \mathtt{Fn}{:}\lambda.\mathtt{p}{:}\mathtt{a}.
\end{array}
$$

$$
\begin{array}{l}
\llbracket \mathtt{x} \mapsto \rho.\rho.\mathtt{k}{:}\mathtt{p}.\mathtt{p}{:}\mathtt{t}.\mathtt{t},\ \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
\quad \rightsquigarrow \llbracket \mathtt{x} \mapsto \llbracket\rrbracket.\rho.\rho.\mathtt{k}(\rho \mapsto \rho.\rho.\mathtt{k}{:}\mathtt{p}){:}\mathtt{t}.\mathtt{t},\ \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{NORHO}}} \\
\quad \rightsquigarrow \llbracket \mathtt{x} \mapsto \bot.\rho.\mathtt{k}(\rho \mapsto \rho.\rho.\mathtt{k}{:}\mathtt{p}){:}\mathtt{t}.\mathtt{t},\ \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DD}}} \\
\quad \rightsquigarrow \llbracket \mathtt{x} \mapsto \bot.\mathtt{k}(\rho \mapsto \rho.\rho.\mathtt{k}{:}\mathtt{p}){:}\mathtt{t}.\mathtt{t},\ \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DD}}} \\
\quad \rightsquigarrow \llbracket \mathtt{x} \mapsto \bot(\rho \mapsto \rho.\rho.\mathtt{k}{:}\mathtt{p}){:}\mathtt{t}.\mathtt{t},\ \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DC}}} \\
\quad \rightsquigarrow \llbracket \mathtt{x} \mapsto \bot{:}\mathtt{t}.\mathtt{t},\ \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
\quad \rightsquigarrow \llbracket \mathtt{x} \mapsto \bot(\rho \mapsto \bot{:}\mathtt{t}),\ \mathtt{k} \mapsto 42.0 \rrbracket.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DC}}} \\
\quad \rightsquigarrow \overbrace{\llbracket \mathtt{x} \mapsto \bot,\ \mathtt{k} \mapsto 42.0 \rrbracket}^{\Pi_{10}}.\mathtt{x} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
\quad \rightsquigarrow \bot(\rho \mapsto \Pi_{10}) \rightsquigarrow_{\text{R}_{\text{DC}}} \\
\quad \rightsquigarrow \bot.
\end{array}
$$



Fig. 7. Normal forms, checked by phino 0.0.161.

$$
\begin{aligned}
&⟦\mathtt{x} ↦ \overbrace{ρ.\mathtt{k}.ρ.\mathtt{t}:\mathtt{t}}^{Π_{11}}, \mathtt{k} ↦ ⟦⟧, \mathtt{t} ↦ ⟦⟦⟧⟧⟧.\mathtt{x}.\mathtt{t} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\quad \rightsquigarrow Π_{11}(ρ ↦ ⟦\mathtt{x} ↦ Π_{11}, \mathtt{k} ↦ ⟦⟧, \mathtt{t} ↦ ⟦⟦⟧⟧⟧).\mathtt{t} \rightsquigarrow_{\text{R}_{\text{SKIP}}} \\
&\quad \rightsquigarrow Π_{11}.\mathtt{t} \rightsquigarrow_{\text{R}_{\text{DOT}}} \\
&\quad \rightsquigarrow ⟦⟧.ρ.\mathtt{k}.ρ.\mathtt{t}(ρ ↦ Π_{11}) \rightsquigarrow_{\text{R}_{\text{NORHO}}} \\
&\quad \rightsquigarrow ⊥.\mathtt{k}.ρ.\mathtt{t}(ρ ↦ Π_{11}) \rightsquigarrow_{\text{R}_{\text{DD}}} \\
&\quad \rightsquigarrow ⊥.ρ.\mathtt{t}(ρ ↦ Π_{11}) \rightsquigarrow_{\text{R}_{\text{DD}}} \\
&\quad \rightsquigarrow ⊥.\mathtt{t}(ρ ↦ Π_{11}) \rightsquigarrow_{\text{R}_{\text{DD}}} \\
&\quad \rightsquigarrow ⊥(ρ ↦ Π_{11}) \rightsquigarrow_{\text{R}_{\text{DC}}} \\
&\quad \rightsquigarrow ⊥.
\end{aligned}
$$

The expressions at Fig. 7 may not be reduced any further; they are in normal form.

B Example of Dataization

Consider an algorithm that converts degrees from Celsius to Fahrenheit:

$$^\circ F = {}^\circ C \times 1.8 + 32,$$

Assuming that $^\circ C$ already equals `25` and is attached to the `c` attribute instead of being provided by a user, the algorithm may be represented by an expression:

$$\begin{array}{l}
[\![\\
\quad \varphi \mapsto [\![\varphi \mapsto \texttt{c.times(1.8).plus(32.0)},\ \texttt{c} \mapsto \texttt{25.0}]\!], \\
\quad \texttt{bytes}(\varphi) \mapsto [\![]\!], \\
\quad \texttt{number}(\varphi) \mapsto [\![\texttt{times}(\texttt{x},\ \rho) \mapsto \texttt{Fn1}{:}\lambda,\ \texttt{plus}(\texttt{x},\ \rho) \mapsto \texttt{Fn2}{:}\lambda]\!] \\
]\!].
\end{array} \tag{6}$$

The evaluations of `Fn1` and `Fn2` are defined as follows:

$$\texttt{Fn1}: \quad \frac{\dfrac{b.\rho \rightsquigarrow n_1}{\langle n_1, e, s\rangle \Downarrow \langle \delta_1, s\rangle} \quad \dfrac{b.\texttt{x} \rightsquigarrow n_2}{\langle n_2, e, s\rangle \Downarrow \langle \delta_2, s\rangle}}{\dfrac{e.\texttt{number}(\varphi \mapsto e.\texttt{bytes}(\varphi \mapsto [\![\Delta \mapsto \delta]\!])) \rightsquigarrow n_3}{\langle b, e, s\rangle \longrightarrow \langle n_3, s\rangle}} \quad \text{if } \delta = \delta_1 \times \delta_2$$

$$\texttt{Fn2}: \quad \frac{\dfrac{b.\rho \rightsquigarrow n_1}{\langle n_1, e, s\rangle \Downarrow \langle \delta_1, s\rangle} \quad \dfrac{b.\texttt{x} \rightsquigarrow n_2}{\langle n_2, e, s\rangle \Downarrow \langle \delta_2, s\rangle}}{\dfrac{e.\texttt{number}(\varphi \mapsto e.\texttt{bytes}(\varphi \mapsto [\![\Delta \mapsto \delta]\!])) \rightsquigarrow n_3}{\langle b, e, s\rangle \longrightarrow \langle n_3, s\rangle}} \quad \text{if } \delta = \delta_1 + \delta_2$$

Figure 8 demonstrates the sequence of transformations that constitutes the dataization of Eq. (6).

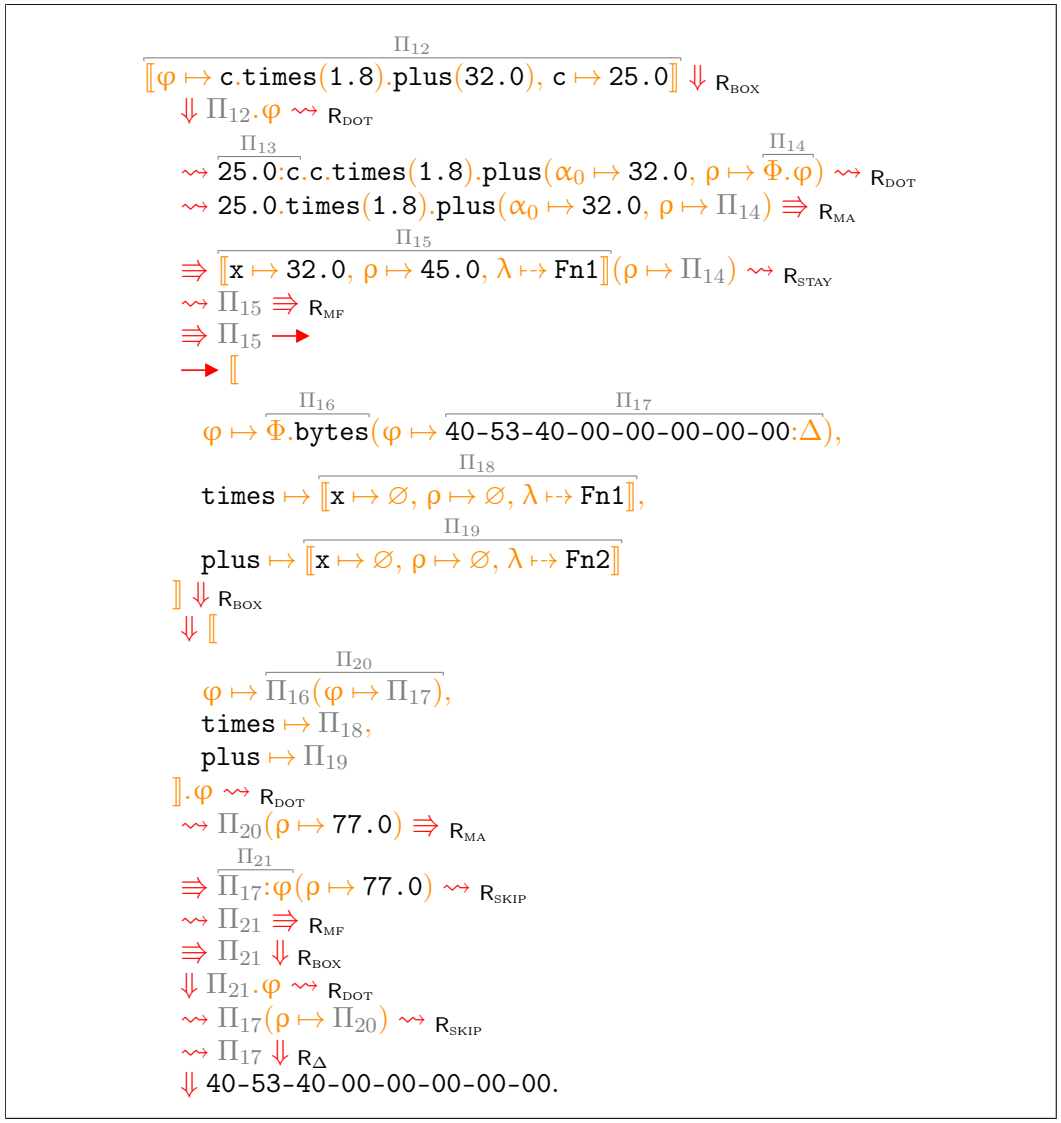


Fig. 8. Dataization of Eq. (6), generated by phino 0.0.161.

C Proofs

This appendix collects the proofs of the theorems stated in Section 5.

Proof of Theorem 5.3. Fix an absolute k and argue by structural induction on the expression e, establishing at once that a derivation of $\langle e, k\rangle \Rightarrow e_1$ exists for some e_1 and that it is finite. By the grammar of Fig. 1 an expression is a locator (Φ or ξ), a terminator $\bot$, a formation, a dispatch, or an application, and its outermost constructor tells these cases apart. The rules of Fig. 2 match the cases one-to-one, so exactly one rule applies to every expression. For Φ, ξ, $\bot$ and a formation $[\![B]\!]$ the

matching rule—R_{CG}, R_{CXI}, R_{CT} or R_{CF}—is an axiom that yields a result in a single step, so the base cases hold; R_{CF} rewrites the formation to itself and never descends into B, halting the recursion at every formation boundary. A dispatch $e_1.\tau$ selects R_{CD}, whose single premise contextualizes the subject e_1; an application $e_1(\tau \mapsto e_2)$ or $e_1(\alpha_i \mapsto e_2)$ selects R_{CA} or R_{CAA}, whose two premises contextualize e_1 and e_2. Every premise is a strict subexpression of its conclusion, so by the induction hypothesis each has a finite derivation and a result, and the chosen rule assembles them into a finite derivation of the whole; the argument k is copied wholesale at R_{CXI} and never traversed. The recursion therefore follows the finite syntax tree and every rule is defined on its case, so contextualization is total on $\mathcal{E} \times \mathcal{K}$ and each derivation terminates. □

Proof sketch of Theorem 5.5. Consider any reduction sequence from a given expression under the rules of Fig. 3; we show it is finite.

Away from R_{DOT} and $\mathsf{R}_{\text{ALPHA}}$, every rule strictly shrinks the term. The *collapsing* rules—$\mathsf{R}_{\text{AMISS}}$, R_{DC}, R_{DCA}, R_{DD}, R_{DL}, R_{MISS}, R_{NULL}, R_{OVER}, $\mathsf{R}_{\text{OVERA}}$, and R_{STOP}—rewrite their redex to the terminator $\bot$, and R_{COPY} and R_{STAY} each discharge one application. $\mathsf{R}_{\text{ALPHA}}$ alone leaves the size unchanged, relabelling a positional argument α_i as the named τ; but it strictly decreases the number of positional arguments, which no rule introduces, so it too fires only finitely often. Hence only finitely many non-R_{DOT} steps occur before either the term is exhausted or a R_{DOT} redex is reached.

It remains to bound the R_{DOT} steps. R_{DOT} replaces a dispatch $[\![B_1, \tau \mapsto n, B_2]\!].\tau$ by the body n contextualized against $[\![B_1, B_2]\!]$ and decorated with the whole formation under ρ. An infinite sequence would need infinitely many such steps, hence a R_{DOT} step that reproduces a dispatch of an attribute on a formation that still binds it—a self-reproducing redex. We argue informally that none arises, though we do not yet exhibit the well-founded measure that would settle it. The body n is contextualized against the formation with the dispatched attribute removed, $[\![B_1, B_2]\!]$ (Section 5.1), so every ξ in n that denotes the enclosing formation becomes a formation binding neither τ nor ρ; a self-reference that reaches for τ again, whether directly as $\xi.\tau$ or through the parent as $\xi.\rho.\tau$, dispatches a missing attribute and collapses to $\bot$ by R_{STOP}, R_{NULL}, R_{MISS}, or R_{DD}. The copy of the formation stored under ρ is reachable only by dispatching ρ, again through such a chain, which dies the same way.

The calculus expresses recursion in only two further ways, and neither is a reduction. Firing the atom that guards a recursive object—the base-case test that decides whether to recur—belongs to evaluation (Section 5.4): a formation carrying an unfired λ-asset matches no rule of Fig. 3 (R_{STOP} excludes it), so the recursive branch never unfolds under normalization. Following the φ-decoration to an inherited attribute belongs to morphing (Section 5.5): no reduction descends into ϕ, so dispatching an absent attribute of a decorated formation is irreducible rather than chasing a possibly cyclic chain.

Every dispatch a term exposes is thus drawn from its finite structure and consumed once, which is why R_{DOT} fires finitely often and the sequence is finite. Every expression therefore has a normal form, unique by Theorem 5.6. This last step is a sketch

rather than a proof: it rests on the informal argument above instead of an explicit well-founded measure, and it is not yet mechanized. Supplying such a measure, and extending the Lean 4 development that certifies Theorem 5.6 to strong normalization, remains future work. □

Proof of Theorem 5.6. We have proved this property mechanically in Lean 4, with the proof available in a public GitHub repository[4]. □

Proof of Theorem 5.10. Unlike normalization, whose rules may rewrite a redex at any position, morphing acts only at the root of its argument n: every rule of Fig. 5 matches the whole of n against e and recurses through its premises on determined subterms. It therefore suffices to show the rules are deterministic—at most one applies to a pair (n, e), and the one that applies fixes the result—for then the relation they define is single-valued, which is the claim. Determinism neither needs nor implies termination, so this holds even though morphing is partial (Section 5.5).

At most one rule matches. By the grammar of Fig. 1, n is a locator (Φ or ξ), the terminator $\bot$, a formation, a dispatch, or an application, and its outermost constructor already separates most rules. For $\bot$ only R_{DEAD} matches and for ξ only R_{ξ}. For Φ the universe decides: R_{Φ} requires $e \neq \Phi$ and R_{MG} requires $e = \Phi$, so exactly one fits a given e. A formation $[\![B]\!]$ with no head operation matches only R_{MF}. A dispatch $m.\tau$ is shared by R_{λ}, R_{φ}, and R_{MD}, which their conditions render disjoint: R_{MD} demands a non-formation subject ($m \notin \mathcal{B}$), whereas R_{λ} and R_{φ} take a formation one, and among those R_{λ} requires an λ-asset while R_{φ} forbids one ($\lambda \notin B$). The formation-headed dispatches left over—a subject with neither an λ-asset nor an φ-attribute, or one already binding τ—match no rule, but they are never normal forms: normalization reduces them by R_{STOP} or R_{DOT} (Section 5.2), and morphing is only ever applied to a normal form, so it never meets them. An application splits four ways on its argument, an exhaustive and disjoint partition: named or positional ($\tau \mapsto a$ versus $\alpha_i \mapsto a$) crossed with absolute or not ($a \in \mathcal{K}$ for R_{MA} and R_{MAA}, $a \notin \mathcal{K}$ for R_{MAD} and R_{MAAD}), and every argument falls in one class.

The matching rule fixes the result. Its meta-variables are pinned by the term: a formation holds at most one λ-asset, so R_{λ} splits $[\![B_1,\ \lambda \mapsto F,\ B_2]\!]$ uniquely, and the subject and attribute of a dispatch, and the subject and argument of an application, are read off the syntax. Each premise is then a single-valued judgment. A normalization premise yields the unique normal form guaranteed by Theorems 5.5 and 5.6. The evaluation premise of R_{λ} calls the function attached to the λ-asset, which by Section 5.4 maps a formation, the universe, and a state deterministically to an expression and a state, and then normalizes that expression to a unique normal form. Every morphing premise is a shorter derivation, single-valued by the induction hypothesis. So the rule maps its configuration, state included, to one result.

By induction on the derivation, then, $\langle n, e, s_1 \rangle \Rrightarrow \langle n_2, s_2 \rangle$ determines n_2 and s_2 from n, e, and s_1: a normal form morphs to at most one formation or $\bot$. □

[4] https://github.com/objectionary/proof

Proof of Theorem 5.12. By the grammar of Fig. 1, every expression, and hence every normal form, is a formation, an application, a dispatch, a locator, or the terminator $\bot$; neither the universe e nor the state s_1 restricts which rule fires. We show that every case other than $\bot$ matches a rule, while $\bot$ matches none.

If $n = \bot$, no rule of Fig. 6 applies, since every rule demands a formation or, for R_{NORM}, an expression distinct from $\bot$. The derivation halts with no datum, which is the error we intend: $\bot$ carries no data to read.

If n is neither a formation nor $\bot$, then $n \notin \mathcal{B}$ and $n \neq \bot$, which is exactly the condition of R_{NORM}; the rule morphs n and dataizes the result, and since morphing delivers a formation or $\bot$ (Section 5.5), that result falls into one of the other cases.

If $n = [\![B]\!]$ is a formation, we inspect its assets and attributes in order: a) if B carries a Δ-asset, R_{Δ} fires; b) otherwise, if B carries an λ-asset, R_{FIRE} fires; c) otherwise, if B carries an φ-attribute, R_{BOX} fires, its condition $[\Delta, \lambda] \cap B = \emptyset$ granted by the two exclusions above; d) otherwise B holds none of Δ, λ, and φ, which is the condition of R_{NONE}; that rule fires and reduces the goal to the dataization of $\bot$, which then halts as above — a formation with nothing to dataize is itself an error. So every formation matches a rule, and $\bot$ alone is left without one.

Termination is by induction on the derivation. R_{Δ} is an axiom and discharges with no premise. Each remaining rule reduces the dataization of n to the dataization of a normal form produced by an operator that itself terminates—contextualization (Section 5.1) and normalization (Section 5.2) for R_{BOX}, evaluation and normalization for R_{FIRE}, and morphing (Section 5.5) for R_{NORM}, while R_{NONE} reduces directly to $\bot$—so every premise is again a dataization judgment, and R_{NORM} never recurs into itself. The derivation is hence finite whenever the recursion through R_{BOX} and R_{FIRE} is well-founded, which holds for every expression that denotes a terminating computation; such a derivation ends either at a datum or at the terminator $\bot$. □

Proof of Theorem 5.13. Dataization is specified by the syntax-directed inference system of Fig. 6, so confluence amounts to the relation $\Downarrow$ being single-valued: whenever $\langle n, e, s_1 \rangle \Downarrow \langle \delta, s_2 \rangle$ and $\langle n, e, s_1 \rangle \Downarrow \langle \delta', s_2' \rangle$, we have $\delta = \delta'$ and $s_2 = s_2'$. We argue by induction on the first derivation, after observing that at most one rule applies to any configuration $\langle n, e, s_1 \rangle$, so both derivations are headed by the same rule.

The conclusions of the five rules partition every configuration, and $\bot$ falls under none of them. If $n = \bot$, no rule matches, so dataization is undefined and there is nothing to disprove. If n is neither a formation nor $\bot$, only R_{NORM} matches. If $n = [\![B]\!]$ is a formation, the asset and attribute it carries select a single rule: a) a Δ-asset selects R_{Δ} and an λ-asset selects R_{FIRE}, the two being mutually exclusive in a normal form; b) with neither asset present, an φ-attribute selects R_{BOX} and its absence selects R_{NONE}, the latter pair separated by the explicit conditions $[\Delta, \lambda] \cap B = \emptyset$ and $[\Delta, \lambda, \varphi] \cap B = \emptyset$. Hence no configuration matches two rules.

Because the same rule heads both derivations, it suffices to check that each rule maps its premises to a single result. R_{Δ} is an axiom whose conclusion reads the result directly off n—the attached data—leaving the state untouched, so it admits no

other outcome. R_{NONE} reduces to the dataization of $\bot$, which is undefined, so it yields no result and hence no second one. R_{BOX} chains three premises: contextualization is a total function (Section 5.1) and yields a unique n; normalization is confluent (Section 5.2) and yields a unique normal form n_1; and the dataization of n_1 is unique by the induction hypothesis. R_{NORM} chains morphing, which is confluent (Section 5.5) and yields a unique formation or $\bot$, with a dataization premise that is unique by the induction hypothesis. R_{FIRE} evaluates the function F attached to the λ-asset. By Section 5.4 a function is a total mapping that deterministically maps formation, universe, and state to an expression and a new state, so its result depends on nothing else. The universe e is fixed throughout the derivation and the state propagates only through the explicit threading of the rules, so both derivations present the call with identical arguments and it returns the same $\langle n, s_2 \rangle$; normalization then yields a unique n_1 and the dataization premise is unique by the induction hypothesis. In every case the preceding premises are determined before the recursive one, so the sub-derivations dataize the same normal form in the same state and the induction hypothesis applies. The two derivations therefore agree on the result, and dataization relates each normal form to at most one datum and state. $\square$